



ROS2 语音&AI与控制

为机器装上嘴巴和耳朵
为机器装上智能辅助系统



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌



目录

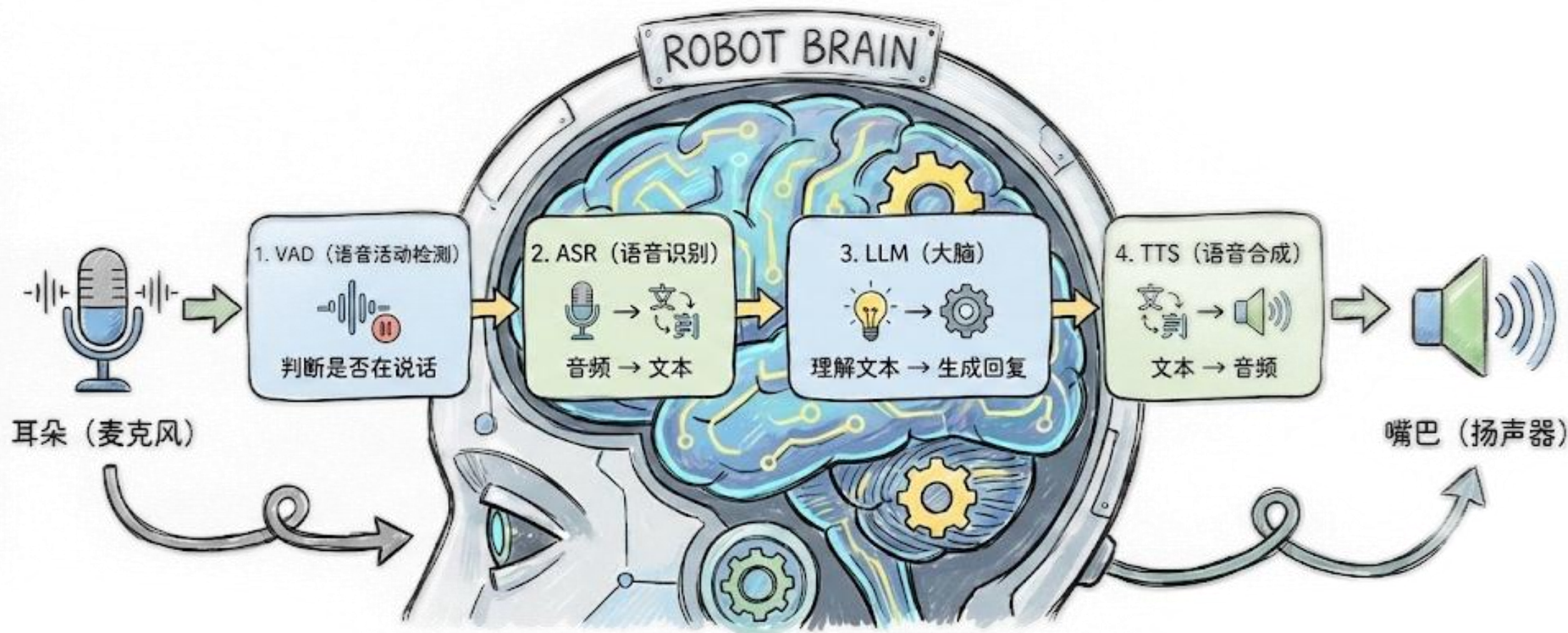
Contents

- ◆ 语音的听和说
- ◆ 大语言模型
- ◆ MCP服务
- ◆ ROS集成



语音的听和说

为什么你的机器人需要“能听会说”？

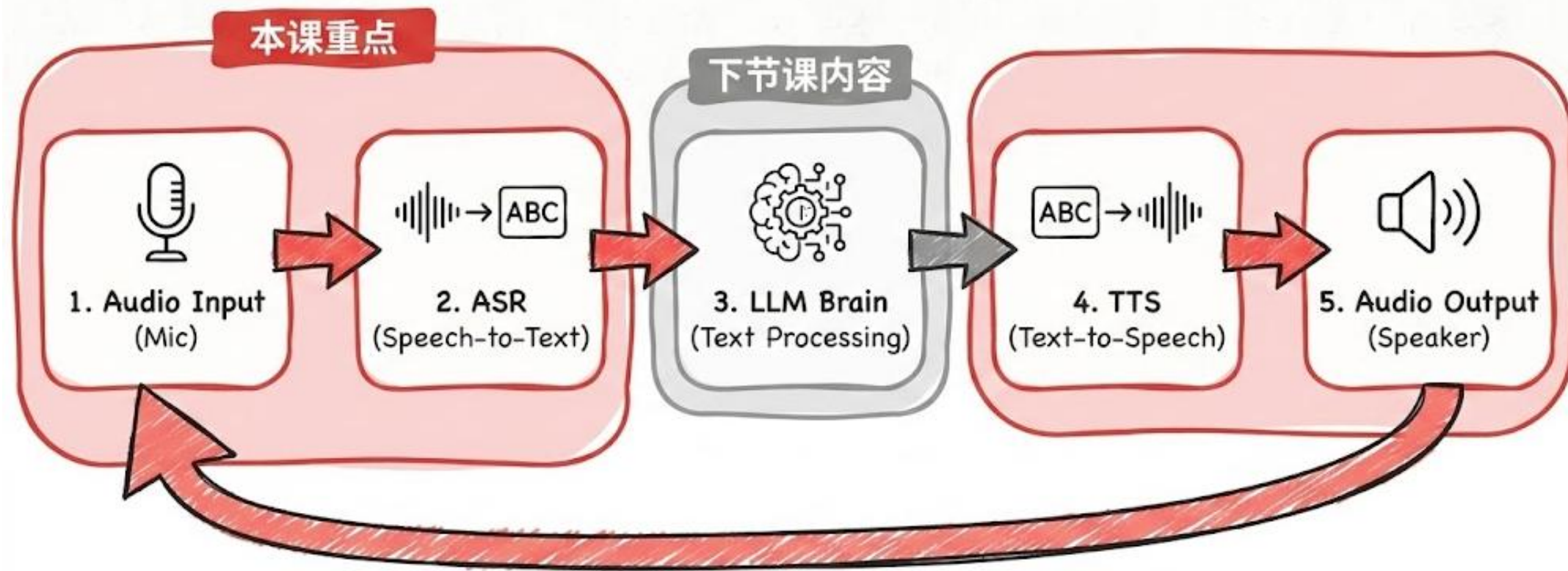


为什么要 VUI? (交互的进化)

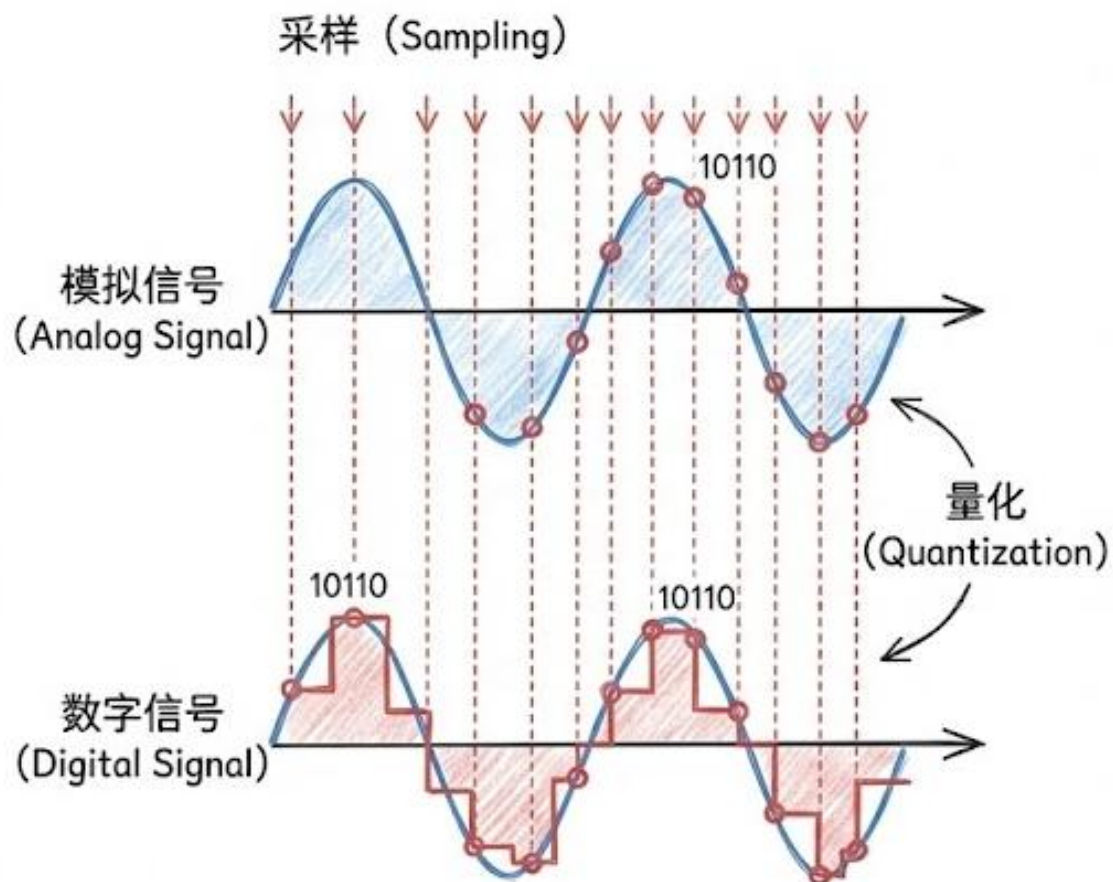


具身智能的核心，是像人一样与物理世界交互。

全链路架构图 (The Loop)



声音的数字化 (ADC 原理)



$$\text{数据量} = \text{采样率} \times \text{量化精度} \times \text{声道数}$$

Sample Rate (采样率)

16000Hz (标准)

44100Hz (CD音质)

Channels (声道)

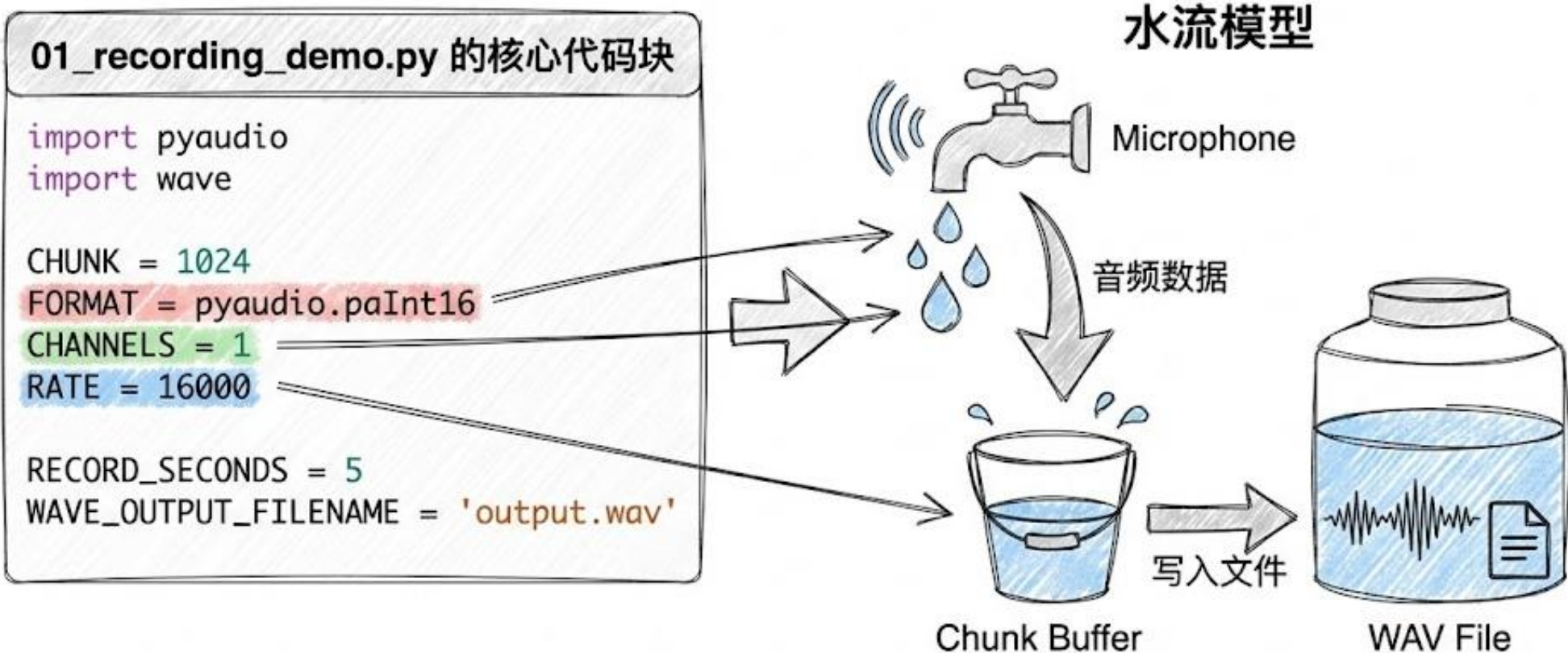


Mono (单声道)



Stereo (立体声)

Python 录音实战 (PyAudio)



环境配置避坑指南 (The Dependency Hell)

ModuleNotFoundError:
No module named 'pyaudio'

Windows:



需下载 .whl
文件离线安装
(附链接)

Linux/Mac:



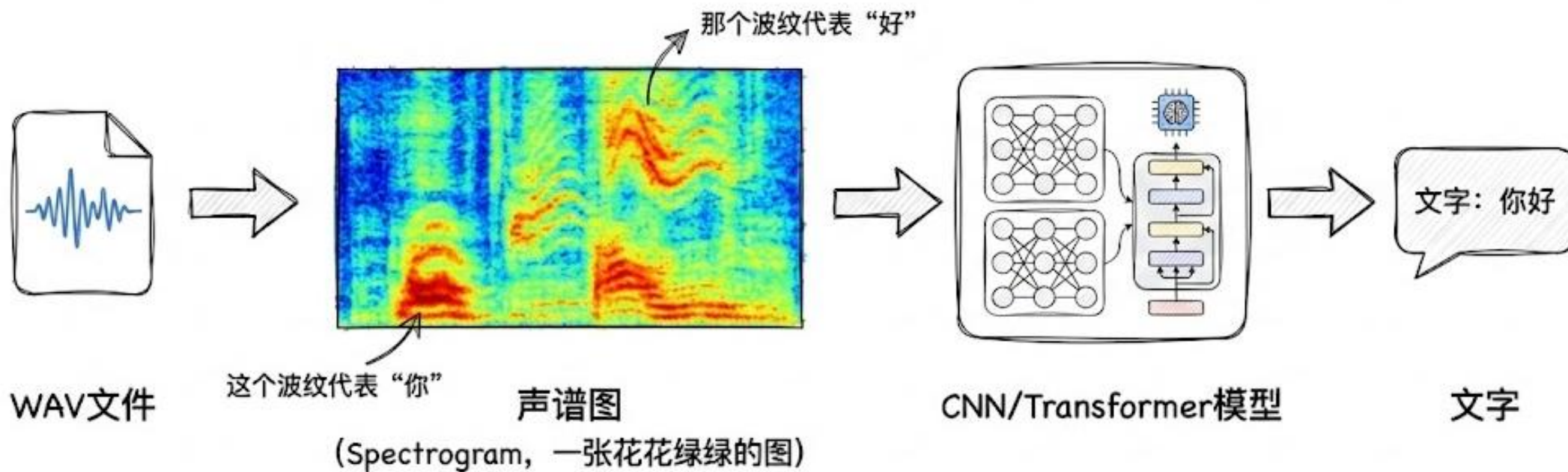
```
sudo apt-get install  
portaudio19-dev
```

万能法 (Universal):



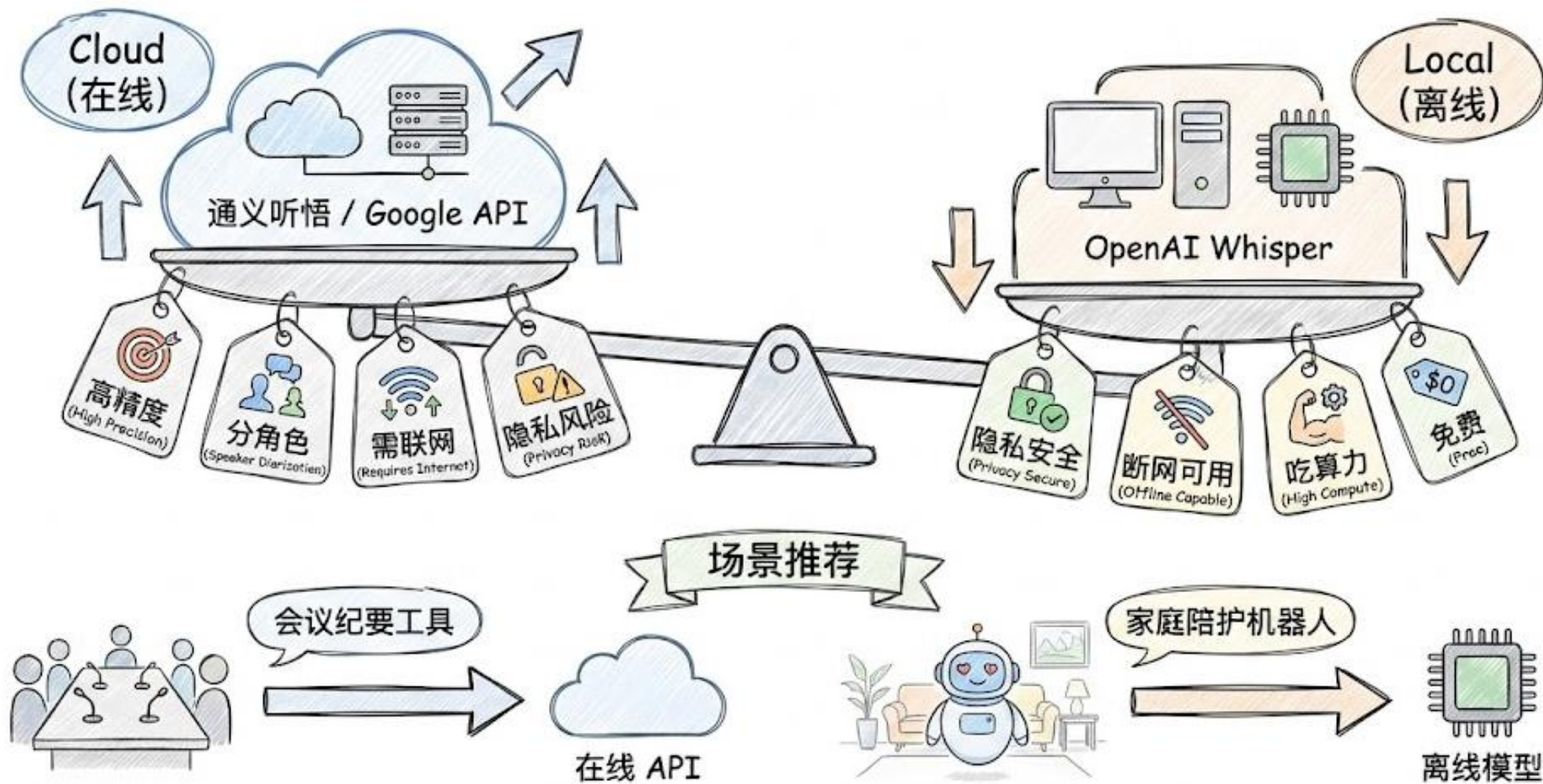
Conda install

机器是如何“听懂”的？(Spectrogram)



“机器听到的不是声音，而是看到了声音的图像。”

技术选型：在线 vs 离线 (Online vs. Local)



Whisper 实战与 FP16 报错修复

```
model = whisper.load_model("base")  
result = model.transcribe("output.wav")  
print(result["text"])
```

RuntimeError:
'LayerNormKernelImpl'
not implemented for 'Half'

result = model.transcribe("output.wav", fp16=False)

修复代码高亮 (第 50 行)



硬核知识点：老旧CPU不支持FP16

小学生算不出来!



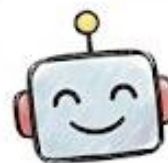
老旧 CPU
(FP32)



微积分
(FP16)



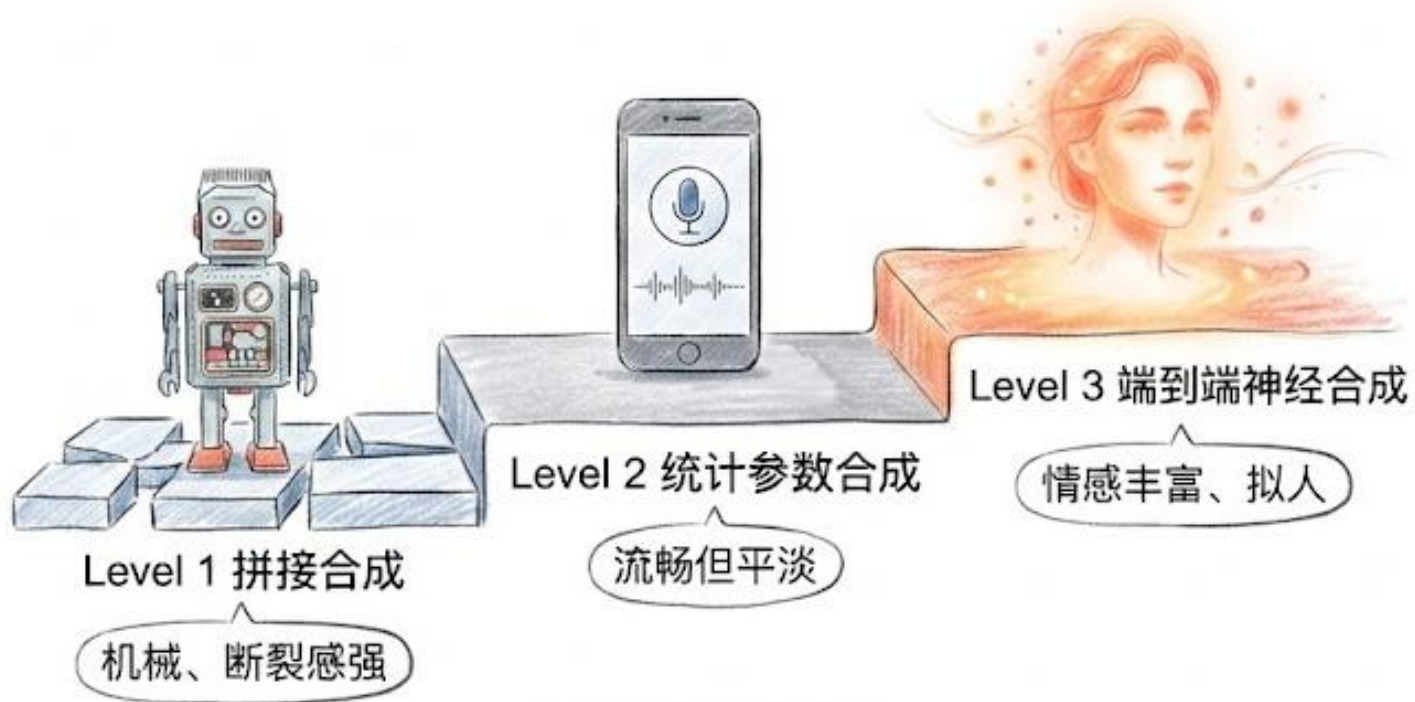
现代 AI
(FP16)



解决方案：告诉模型
“不赶时间，求稳不求快，
用 FP32 慢慢算”。



TTS 的技术进化



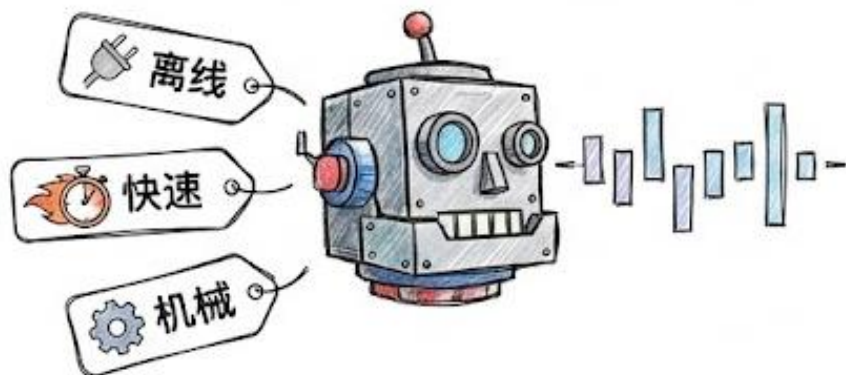
音频波形对比



离线与在线 TTS 实战 (Edge-TTS)

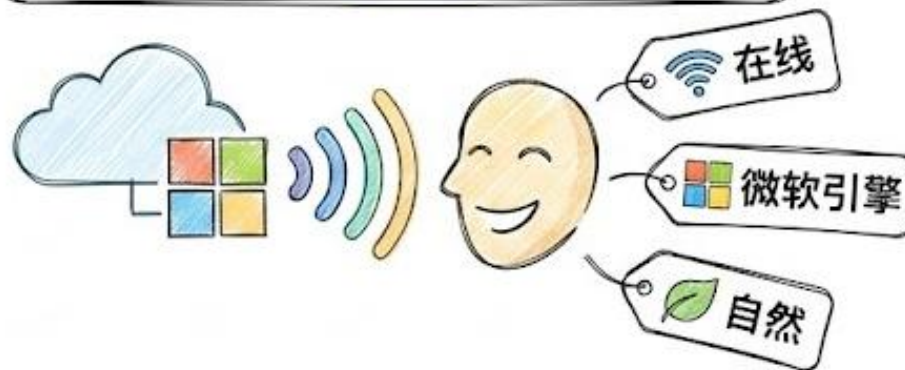
离线 TTS (Offline) - pyttsx3

```
import pyttsx3  
  
engine = pyttsx3.init()  
engine.say("hello") # 核心代码  
engine.runAndWait()
```



在线 TTS (Online) - Edge-TTS

```
import edge_tts  
  
text = "你好, 世界"  
# 核心代码  
communicate = edge_tts.Communicate(text)  
await communicate.save("out.mp3")
```



报警指令
用离线

应用建议

闲聊对话
用在线

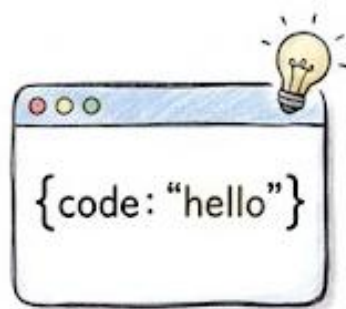




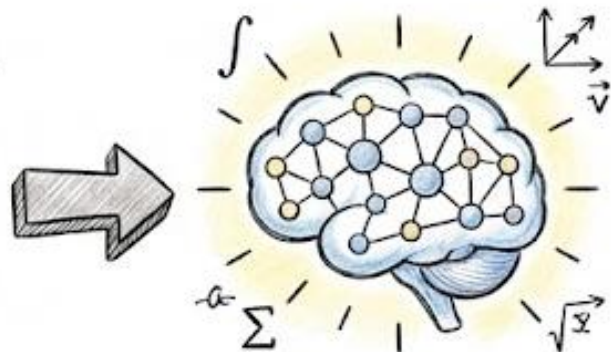
大语言模型

AI 为什么能听懂你的话?

从 V4 版“玩具”代码到大模型原理



“玩具”代码

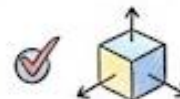


大模型原理

【学习目标】



理解传统音箱的“智障”原因（规则匹配）



掌握大模型核心：Embedding（词向量）



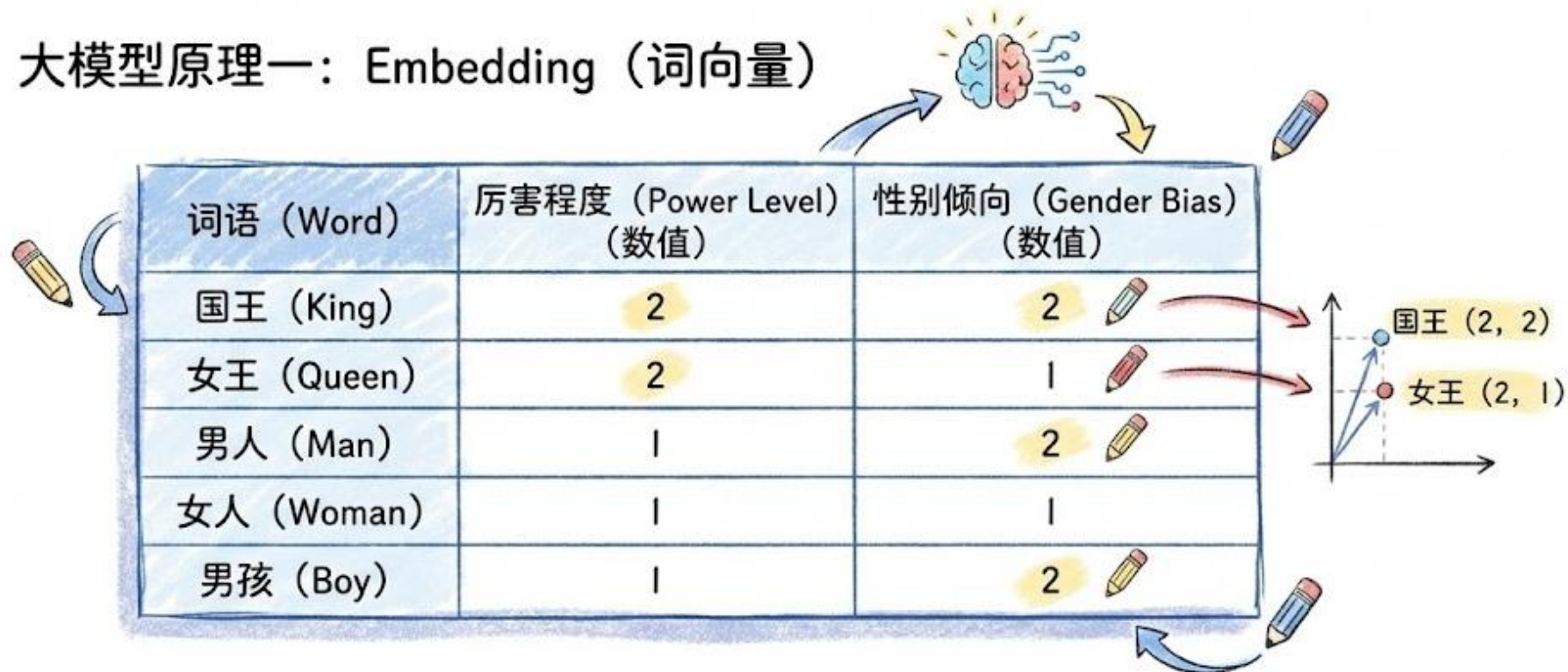
掌握大模型机制：概率预测（文字接龙）

传统 vs 现代（激活旧知）



万物皆数（原理拆解）

大模型原理一：Embedding（词向量）



注脚：数据来源 = 机器学习阅读海量文本自动生成

词语之间的代数运算

难点突破：词语的数学运算

$$\text{国王}(2,2) - \text{男人}(1,2) + \text{女人}(1,1) = ?$$

厉害程度
(Power Level)

$$2 - 1 + 1 = 2$$



性别倾向
(Gender Bias)

$$2 - 2 + 1 = 1$$



(2, 1)

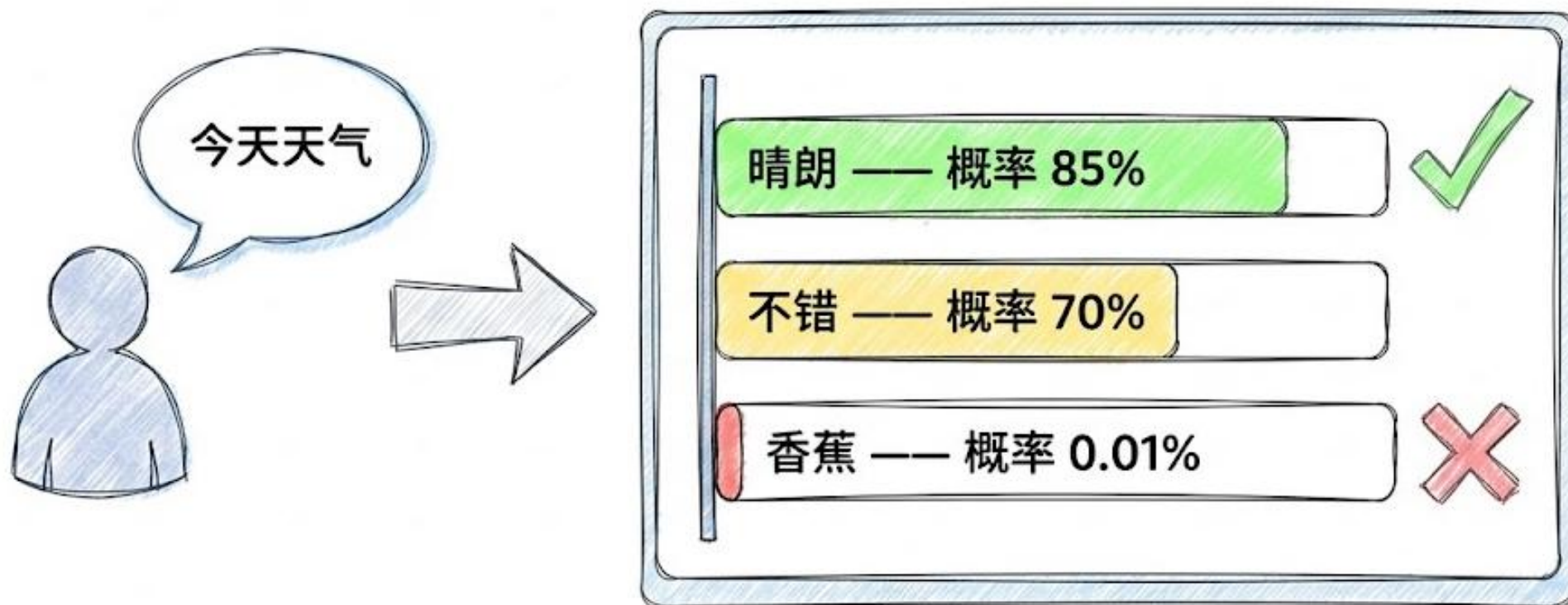


女王
(Queen)

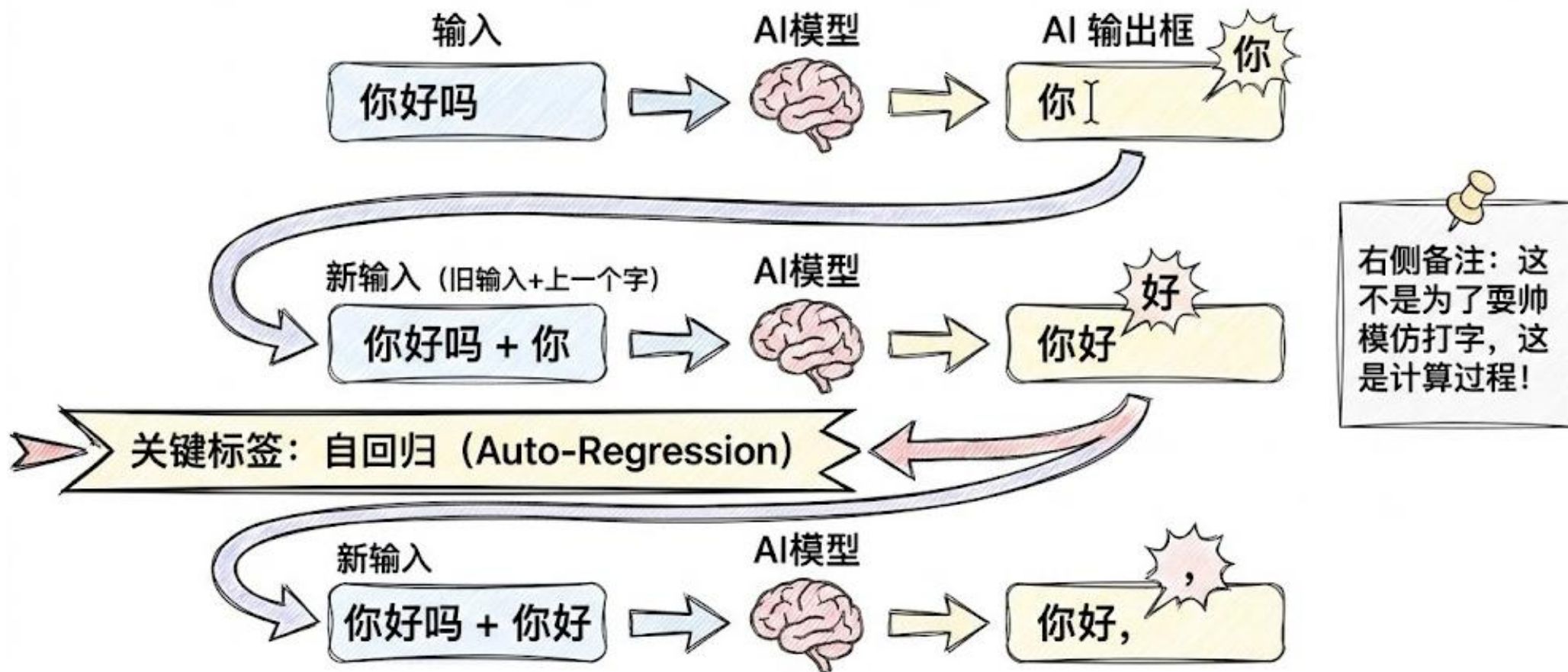
结论：计算机通过计算数字关系，理解了词义关系。

大模型原理二：概率预测（Next Token Prediction）

基于上文推理下文，而非死记硬背。



为什么 AI 是一个字一个字蹦出来的?



API: 连接我们与大模型的桥梁



顾客 (你/Python代码)

服务员 (API)

厨房 (大模型服务器)

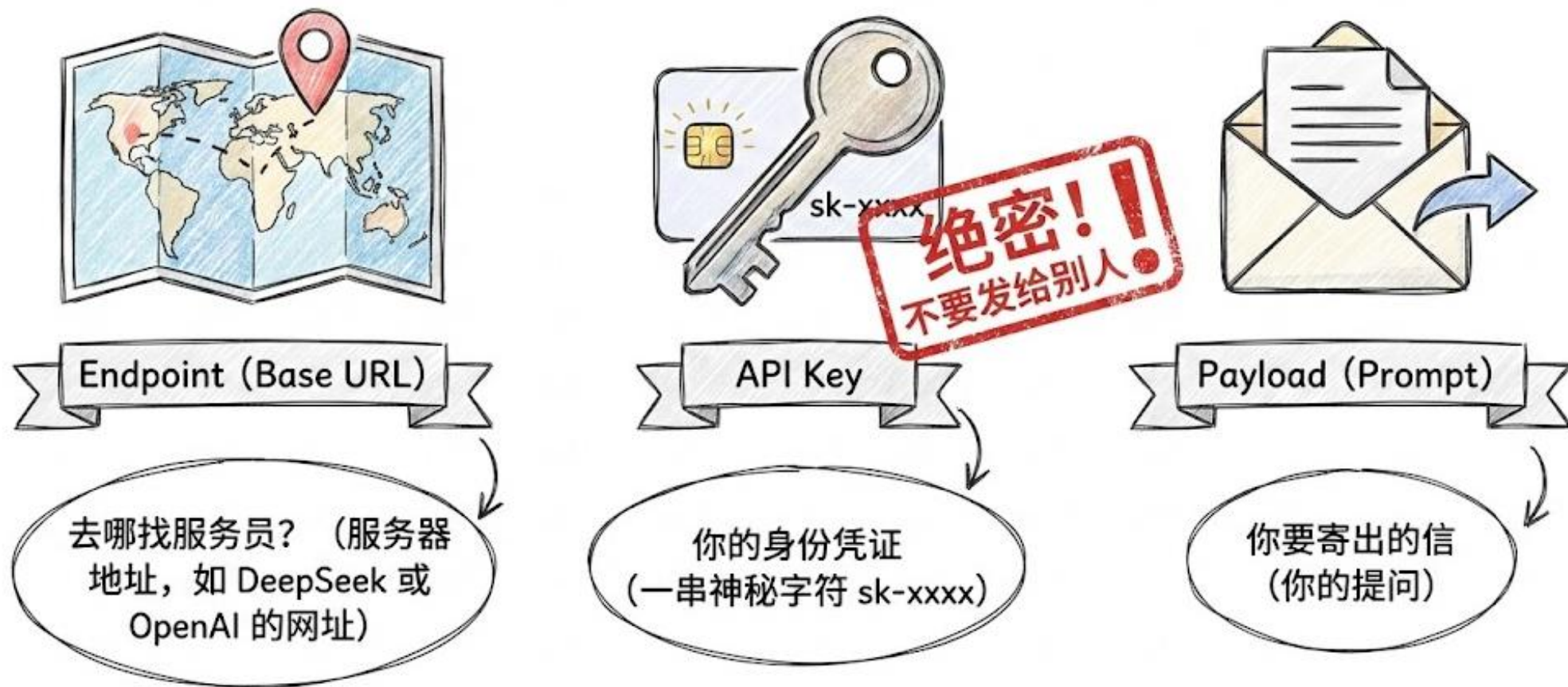
你不能直接进厨房炒菜。

你需要通过服务员 (API) 传话。

API = Application Programming Interface。



调用大模型的三把“钥匙”



最简单的调用代码 (Python)

```
# 1. 引入库
from openai import OpenAI
# 2. 配置客户端 (服务员)
client = OpenAI(
    api_key="sk-你的密钥",
    base_url="https://api.deepseek.com"
)
# 3. 发送请求 (点菜)
response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "user", "content": "你好，你是谁? "}
    ]
)
# 4. 打印结果 (上菜)
print(response.choices[0].message.content)
```





MCP服务

为什么大模型需要 MCP?



大模型的局限

虚拟空间的智能，无法直接干预物理世界。

传统方案

硬编码适配 (Hard-coding)，每加一个设备都要写大量冗余代码。

MCP 方案

像 USB 一样即插即用，定义统一的“上下文协议”。

什么是 MCP?

Model Context Protocol (模型上下文协议)

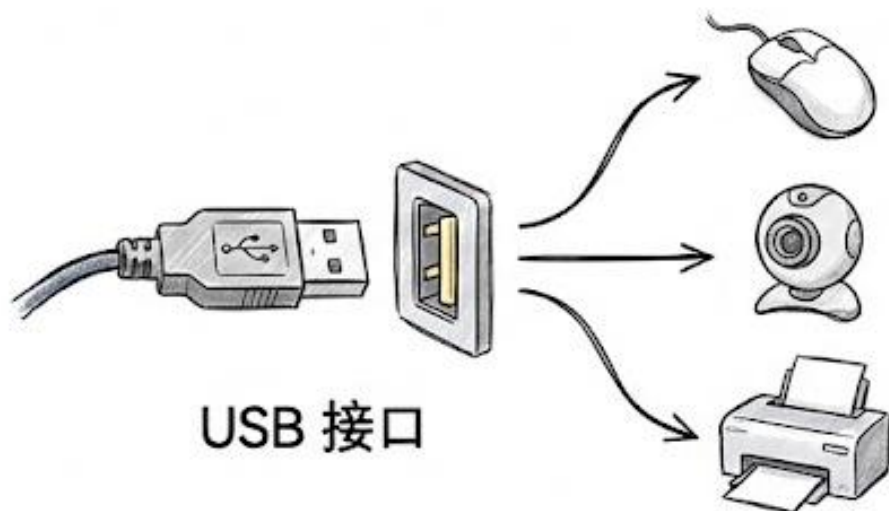
★ 关键点：由 Anthropic 公司在 2024 年底发布，极具前瞻性。



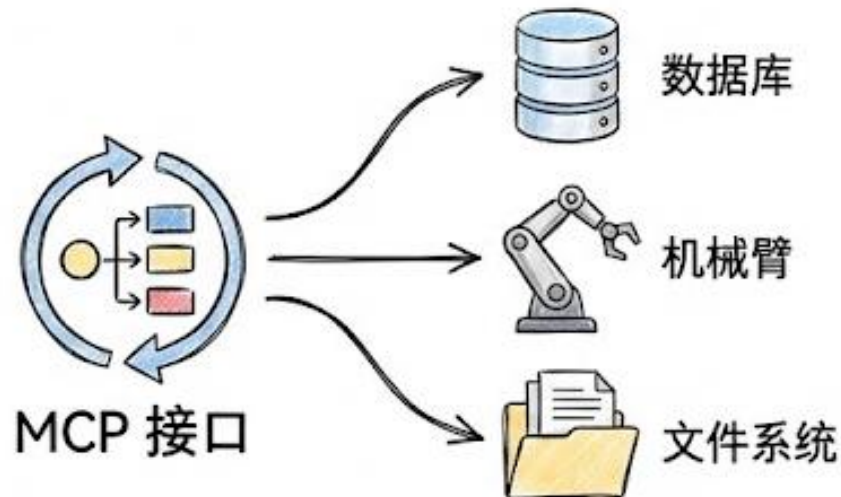
预警：大模型由于训练数据截断，
无法回答最新的 MCP 细节。



核心机制：MCP 与 USB 的类比



标准接口 + 各种外设（鼠标、摄像头、打印机）



标准协议 + 各种服务器（数据库、机械臂、文件系统）

作用说明：解耦（Decoupling）

让大模型不再关心底层硬件逻辑，只关心“接口清单”。



实战演练：编写你的第一个 MCP Server

 环境准备

```
pip install fastmcp
```

Server 核心三步走：



本节小结

★ 重点 (Key Points)

-  MCP 的 C/S 架构
-  @mcp.tool 的使用

? 难点 (Challenge Points)

- 
- 理解模型如何通过协议感知外部工具

📚 分层作业



基础题 (Basic)

成功运行 Demo，并给 Server 增加一个 multiply (乘法) 工具。



提高题 (Advanced)

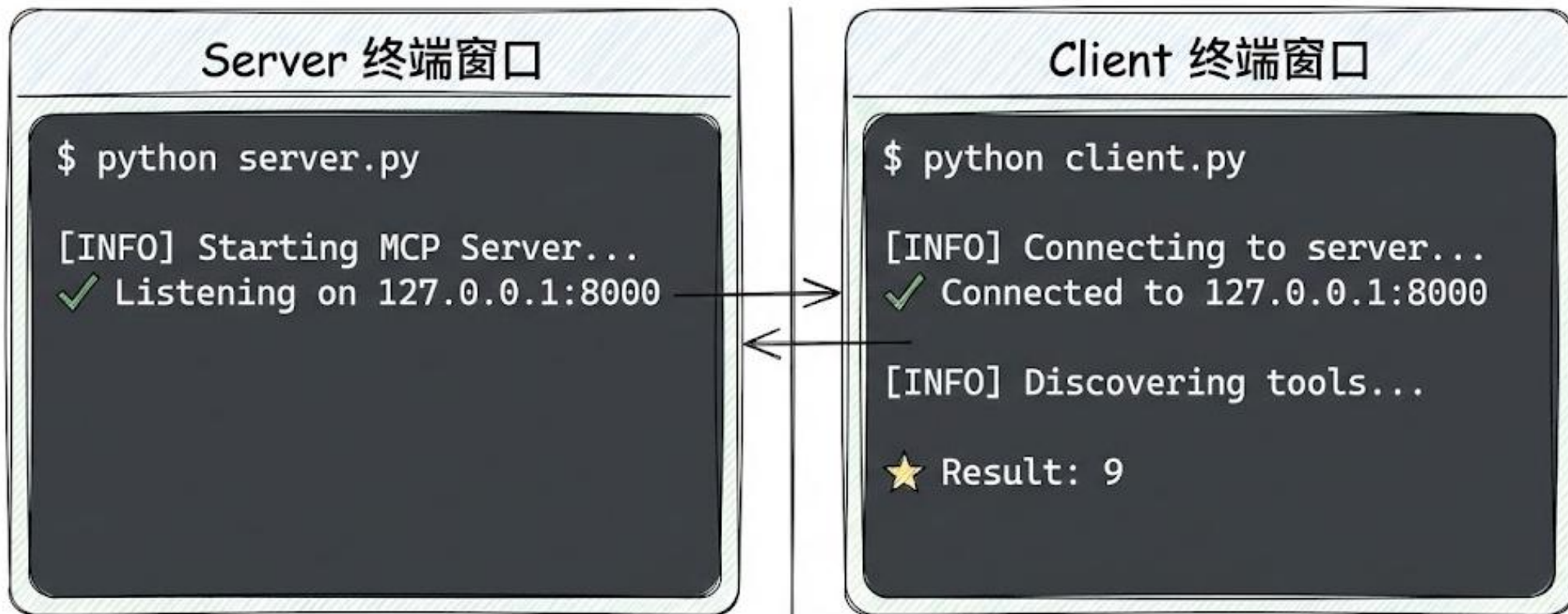
尝试写一个读取本地磁盘剩余空间的 MCP 工具。



拓展题 (Extension)

思考如果要把家里的智能灯接入 MCP，你需要定义哪些 tool?

双端联调实战（让代码跑起来）



字幕提示：重点观察：Client 连接瞬间发生的“握手”与“工具发现”

原理深挖（透过现象看本质）

核心概念：解耦（Decoupling）与 分布式（Distributed）

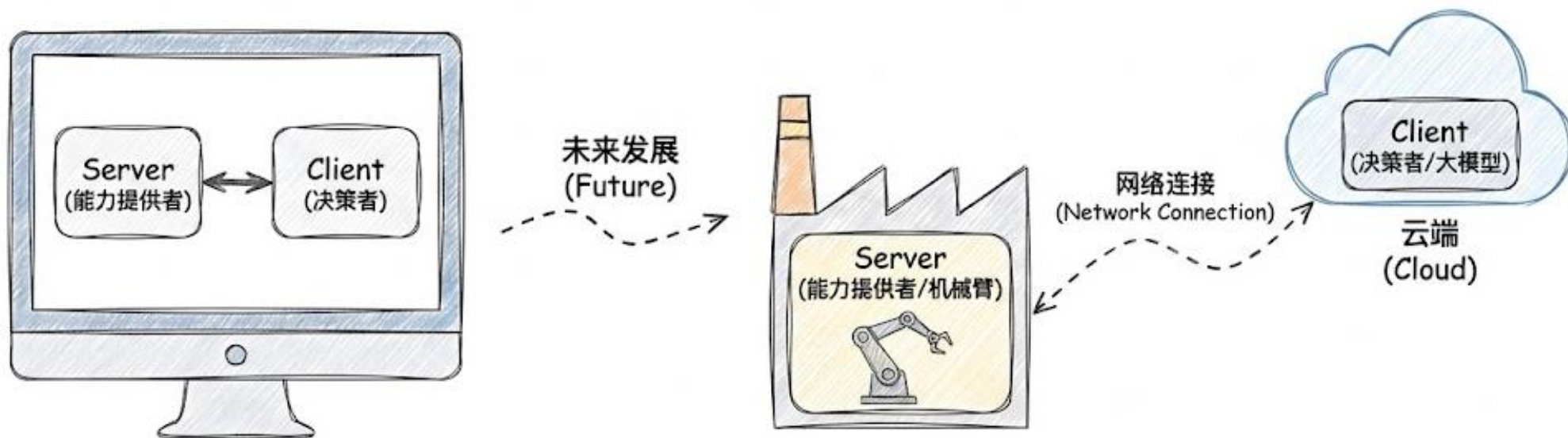


图 A: Server 和 Client 在同一台电脑（现状）

图 B: Server 在工厂（机械臂），Client 在云端（大模型），中间通过网络连接。

文字：Server = 能力提供者（硬件/工具）；Client = 决策者（大模型）。

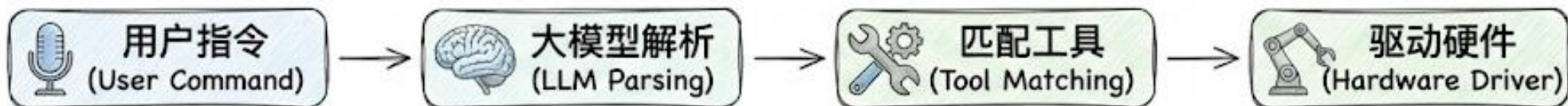
MCP 协议实战（智能硬件）



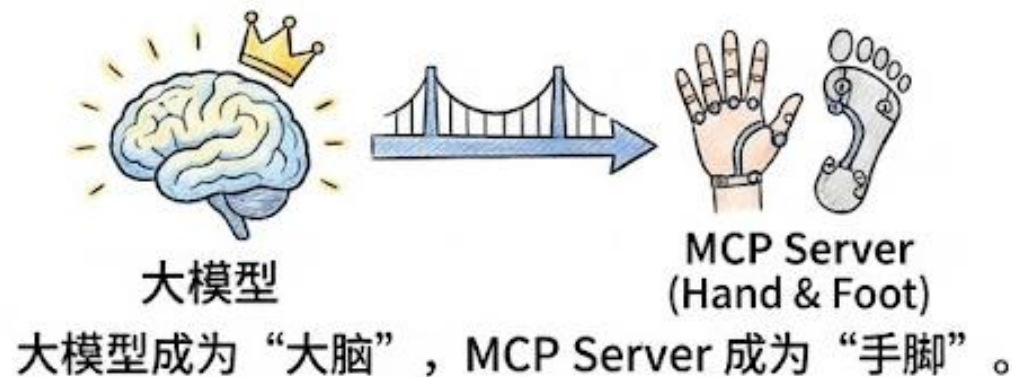
工具列表（Tools List）

- twist_waist()（扭腰）
- nod_head()（点头）
- lift_head()（抬头）

交互流程：



本节小结

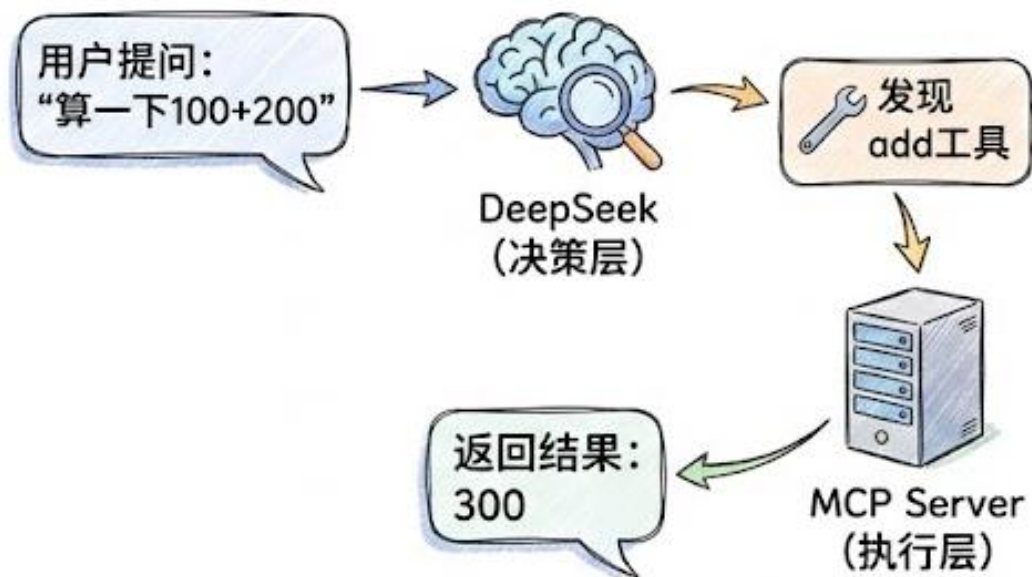


如果我想用大模型控制家里的智能灯泡，我的MCP Server 应该提供哪些工具？

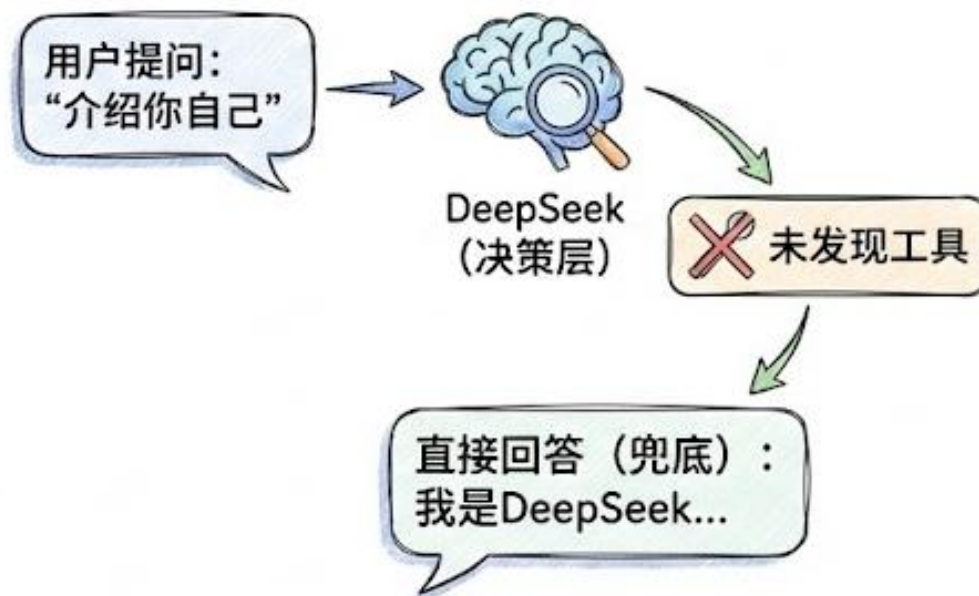
-  开灯
-  关灯
-  调颜色

LLM + MCP = 智能大脑 + 灵活手脚

场景一：工具调用 (Tool Use)



场景二：直接回答 (Direct Answer)

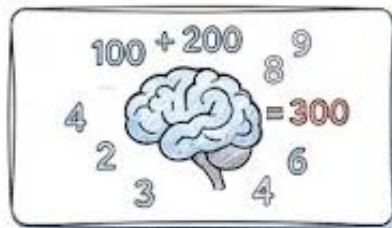


重点：强调“路由”和“分发”的概念

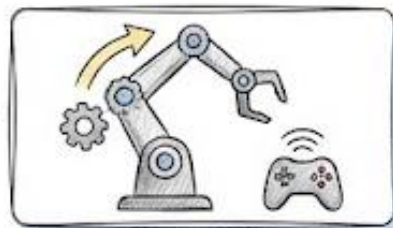
实战升级——构建“机械臂”服务器



场景切换 (Scene Switch)



“虚拟算术”



“物理控制”

任务卡

构建 MCP_Robot_Server



rotate_waist (腰部旋转) | 范围 [-90, 90]



control_gripper (夹爪控制) | 范围 [0, -90]



技巧:

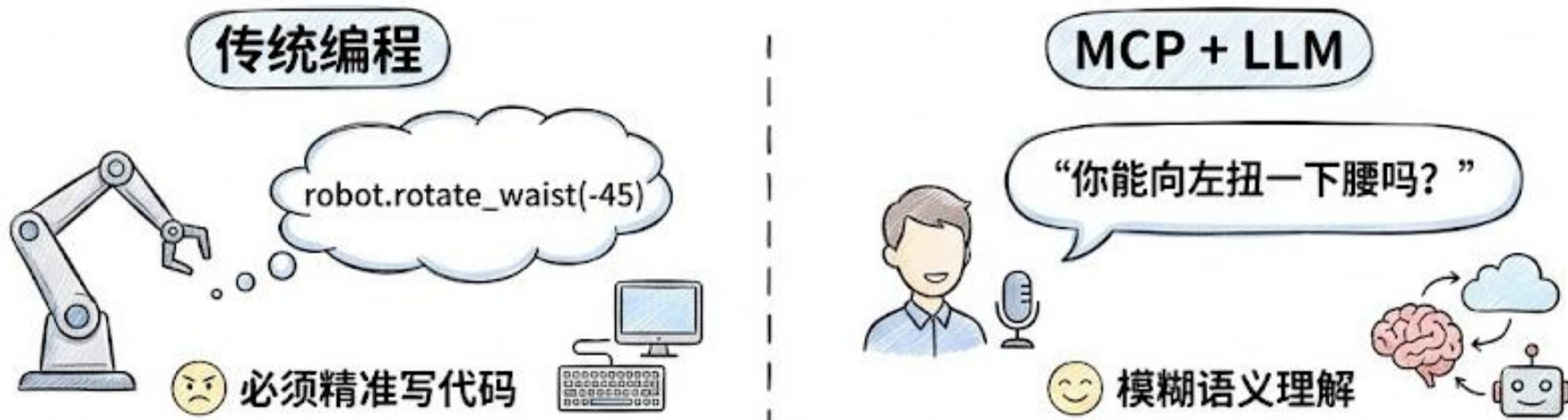
AI 辅助编程 (Prompt Engineering)

AI, help me write...

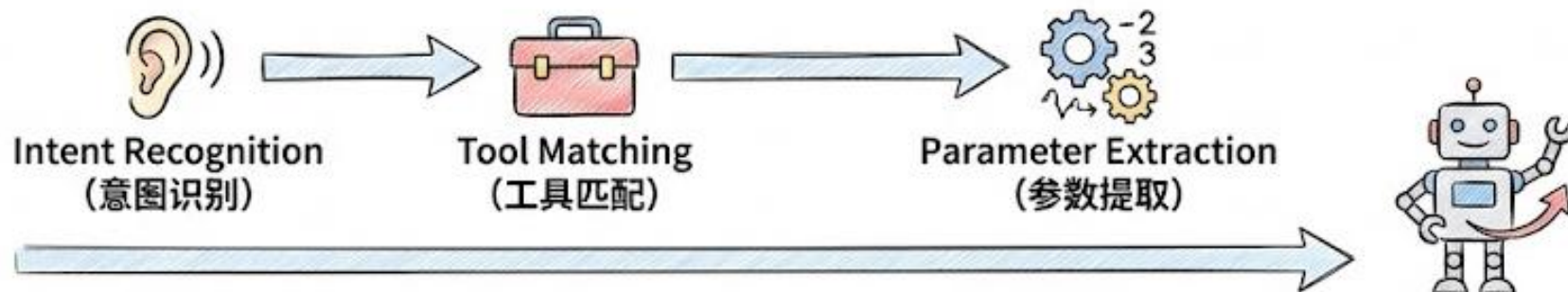


自然语言的魔力——“扭一下腰”

对比展示



核心原理



见证奇迹 & 未来展望

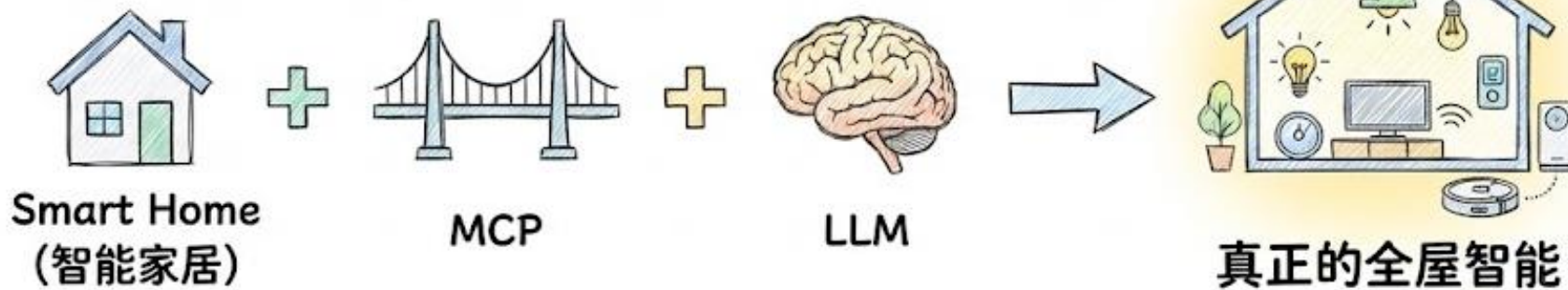


执行结果 (Execution Results)

```
[Server 日志] Calling tool: rotate_waist with args: {"angle": -90}  
[Server 日志] Calling tool: control_gripper with args: {"angle": -45}
```



未来愿景 (Future Vision)

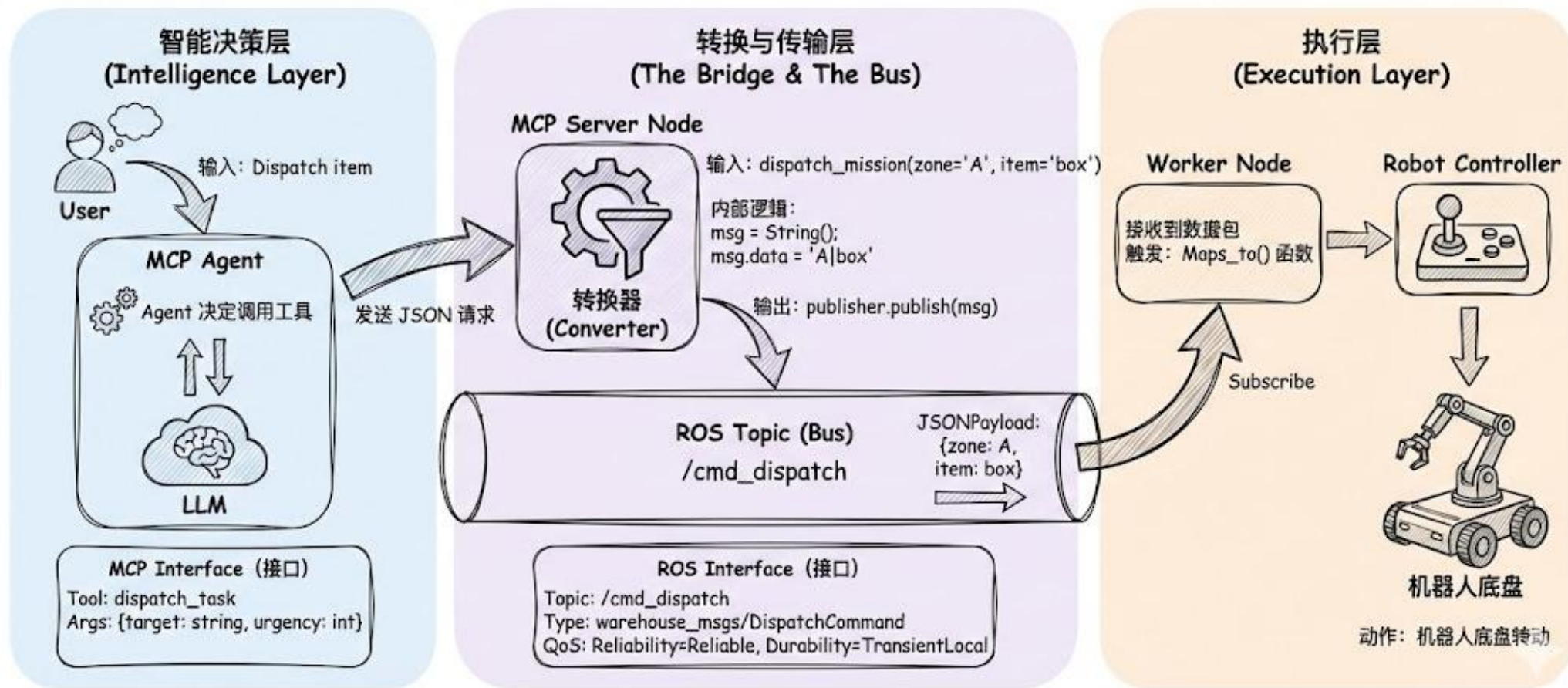




ROS集成

System Architecture: The MCP-to-Topic Bridge

全流程架构图 (The Full Pipeline)





传智教育旗下高端IT教育品牌